Manuel développeur

PROJET TETRIS

Adrien LABAYLE, Thomas MAS
Polytech Clermont | IMDS_4A

1. Sommaire

Manuel développeur

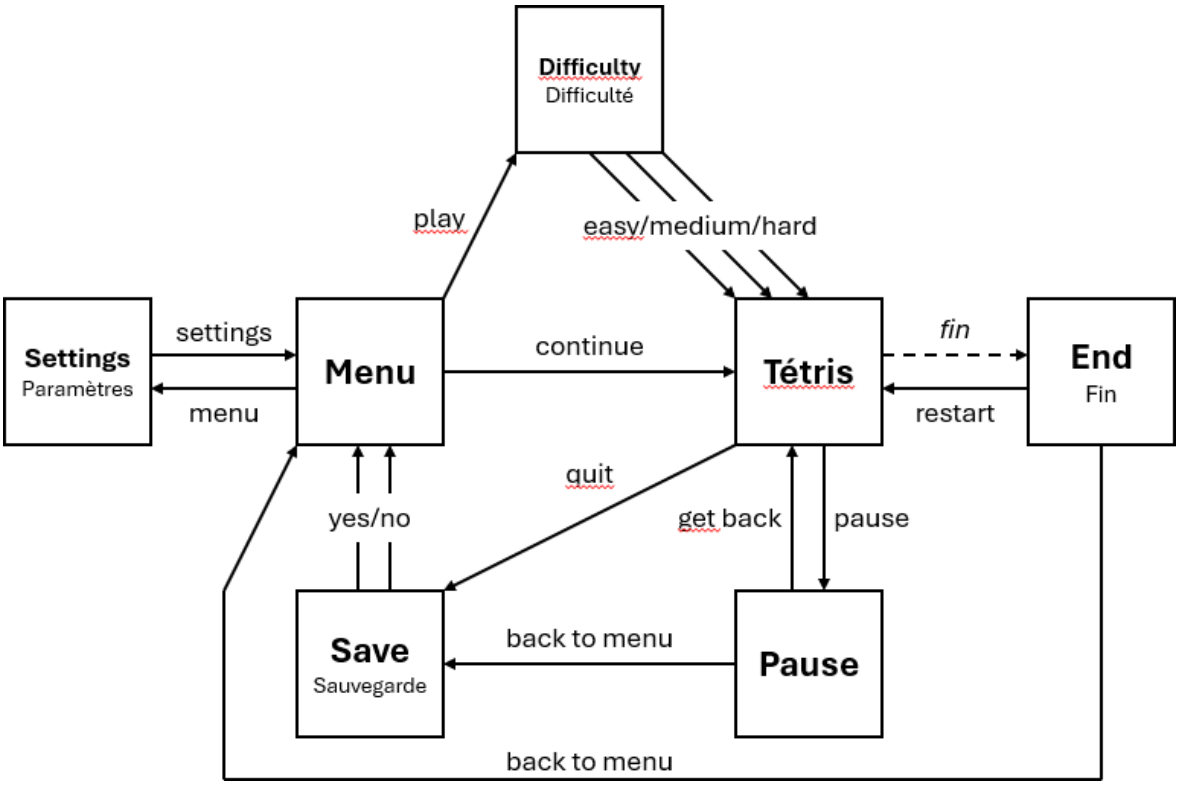
1. Sommaire 1
Partie 2 : Analyse et conception du logiciel 2
2.1. Cas d'utilisation 2
2.2. Diagrammes de classes 4
2.2.1. Classe « MainWindow » 5
2.2.2. Classe « Game » 5
2.2.3. Classe « Grid » 6
2.2.4. Classe « Tétromino » 6
Partie 3 : Programmation du logiciel 7
3.1. Arborescence du projet 7
3.1.1. Constructeurs et destructeurs 7
3.2. Structure de données et représentation 8
3.2.1. Gestionnaires d'évènements 8
3.3. Points importants du code 9
3.3.1. Gestion des collisions 9
3.3.2. Gestion de l'intégration 9
3.3.3. Gestion des suppressions de lignes 10
3.3.4. Gestion de l'ombre 10
3.3.5. Gestion des sauvegardes 11

Partie 2 : Analyse et conception du logiciel

2.1. Cas d'utilisation

L'application possède 7 écrans :

- **Menu** : Cet écran apparaît en premier. L'utilisateur a trois options :
    - Démarrer une nouvelle partie avec le bouton « play ».
    - Reprendre une partie en cours avec le bouton « continue ».
    - Lire les règles du jeu avec le bouton « settings ».
- **Difficulty** : Cet écran placé entre « Menu » et « Tétris » permet de choisir entre les trois niveaux de difficulté (easy, medium ou hard). Le niveau choisi s'applique aux parties successives.
- **Tetris** : Il s'agit de l'écran principal du jeu. Y sont affichés la grille de jeu dans laquelle évolue le tétromino actuel, le tétromino suivant, le score actuel et la durée de la partie. A intervalles réguliers (dont la longueur dépend de la difficulté choisie), le tétromino actuel descend d'une ligne dans la grille. Le joueur peut le faire tourner et le déplacer horizontalement avec les flèches du clavier. Il peut également accélérer sa chute vers le bas (ligne par ligne avec la flèche du bas ou instantanément avec la touche espace), car l'écran de jeu affiche également l'ombre du tétromino actuel, pour que le joueur sache où il va atterrir. Quand le tétromino atteint le bas de la grille, les lignes complètes sont effacées, on recalcule le score, le tétromino actuel est supprimé, le tétromino suivant le remplace en haut de la grille, et un nouveau tétromino suivant est généré. Si la pile de tétrominos atteint le sommet de la grille, la partie s'arrête et l'écran de fin jeu s'affiche. Enfin, l'utilisateur peut :
    - Mettre le jeu en pause avec le bouton « pause ».
    - Revenir au menu avec le bouton « quit ».
- **Pause** : Cet écran est similaire à l'écran « Tétris », sauf que le mouvement et les rotations de tétromino sont impossibles. L'utilisateur peut :
    - Reprendre la partie avec le bouton « get back » (retour).
    - Revenir au menu avec le bouton « back to menu ».
- **End** (Fin) : Cet écran se déclenche automatiquement en cas de « game over ». Il est similaire à l'écran de pause avec les différences suivantes :
    - Au lieu de reprendre la partie, on peut en démarrer une autre avec le bouton « restart ».
    - On peut aussi revenir au menu avec le bouton « back to menu », mais sans passer par l'écran de sauvegarde.
- **Save** (Sauvegarde) : Cet écran apparaît lorsqu'on revient au menu depuis l'écran « Tetris » ou « Pause ». Il permet à l'utilisateur de choisir de sauvegarder ou non la partie en cours pour pouvoir la reprendre depuis l'écran « Menu » avec le bouton « restart ».
- **Settings** (Paramètres) : Cet écran affiche les règles du jeu.



Relations entre les différents écrans

2.2. Diagrammes de classes

Il existe 4 classes principales :

- **`mainwindow`**
- **`game`**
- **`grid`**
- **`tetromino`**

La classe « Tetromino » comprend elle-même 7 sous-classes :

`I_tetro`, **`J_tetro`**, **`L_tetro`**, **`O_tetro`**, **`S_tetro`**, **`T_tetro`**, **`Z_tetro`**

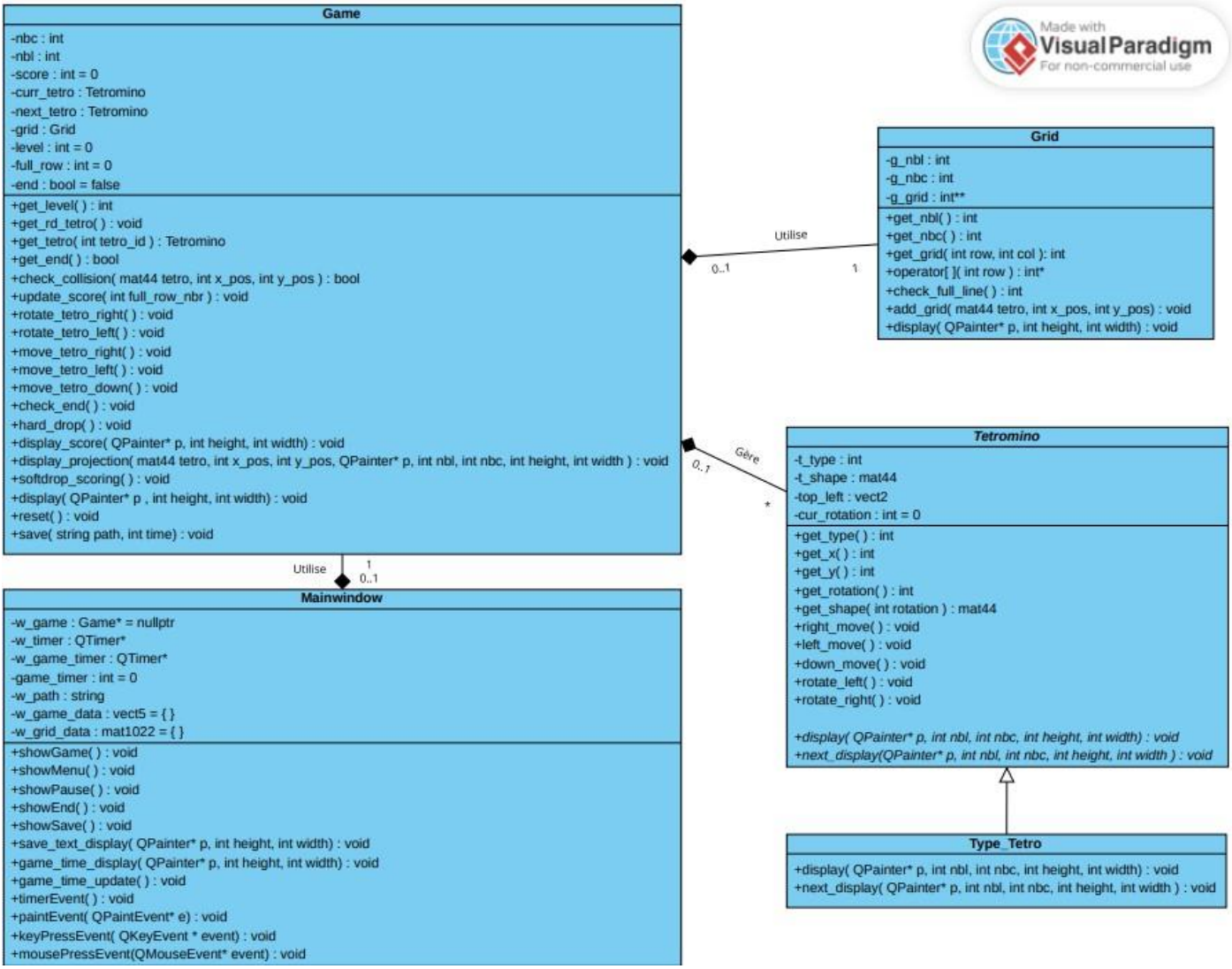


Diagramme de classes détaillé

2.2.1. Classe « MainWindow »

Il s'agit d'une classe technique qui gère :

- L'affichage des différents écrans de l'application
  - **showGame**
  - **showMenu**
  - **showPause**
  - **showEnd**
  - **showSave**
  - **showDifficulty**
  - **showSettings**
- L'interaction avec les boutons (voir la section 3.2.1 pour plus de détails)
  - **on_play_button_clicked**
  - **on_quit_button_clicked**
  - **on_pause_button_clicked**
  - **on_get_back_button_clicked**
  - **on_back_to_menu_button_clicked**
  - **on_restart_button_clicked**
  - **on_back_to_menu_end_button_clicked**
  - **on_no_button_clicked**
  - **on_yes_button_clicked**
  - **on_continue_button_clicked**
  - **on_easy_button_clicked**
  - **on_medium_button_clicked**
  - **on_hard_button_clicked**
  - **on_settings_button_clicked**
  - **on_menu_button_clicked**
- L'interaction avec les touches du clavier (**keyPressEvent**)
- L'interaction avec les touches de la souris (**mousePressEvent**)
- Les événements chroniques (**timerEvent**)

2.2.2. Classe « Game »

Cette classe représente une partie du jeu « Tetris ». On génère un objet de cette classe au début d'une partie (bouton `play`), et on le supprime à la fin (boutons `quit` et `back to the menu`).

Elle implémente les aspects fonctionnels du jeu, tels que :

- La génération aléatoire des tétrominos parmi les sept possibles (**get_rd_tetro**)
- La rotation et le mouvement des tétrominos dans la grille
  - **rotate_tetro_right**
  - **rotate_tetro_left**
  - **move_tetro_right**
  - **move_tetro_left**
  - **move_tetro_down**
  - **hard_drop**
- Les collisions (**check_collision**)
- Le calcul et l'affichage du score (**update_score**)
- L'affichage du score et de l'ombre du tétromino

- **`display_score`**
- **`display_projection`**

2.2.3. Classe « Grid »

Cette classe représente la grille dans laquelle se déroule le jeu sous la forme d'un tableau en deux dimensions. Elle possède aussi des accesseurs (**`get_nbl`**, **`get_nbc`**, **`get_grid`**). Elle s'occupe notamment de :

- La détection et la suppression des lignes complètes, ainsi que la « descente » du reste de la grille (**`check_full_line`**)
- L'intégration des tétrominos arrivés en bas de la grille (**`add_grid`**)

2.2.4. Classe « Tétromino »

Cette classe représente les tétrominos, des dominos de quatre cases qui sont à la base du principe du jeu Tetris. Il en existe sept types différents, chacun représenté par une sous-classe. Chaque tétromino possède les attributs suivants :

- Son identifiant parmi les 7 types (**`t_type`**)
- Sa position x et y dans la grille de jeu, ou plutôt celle du coin supérieur gauche (**`top_left`**)
- Sa rotation actuelle (**`curr_rotation`**)
- Un tableau de dimension 4×4×4 (**`t_shape`**), qui représente la forme du tétromino dans un tableau 4×4 pour les 4 rotations possibles. Les cases qui appartiennent au tétromino prennent la valeur **`t_type`**, et les cases vides prennent la valeur 0.

0	T	0	0
0	T	T	0
0	0	T	0
0	0	0	0

Exemple (tétromino S, rotation 0)

La classe comprend des accesseurs (**`get_x`**, **`get_y`**, **`get_rotation`**) et également des fonctions de mouvement et de rotation. Ce ne sont pas des doublons inutiles des fonctions de la classe « Game », car elles sont appelées uniquement lorsque le test de collision est négatif.

- **`right_move`**
- **`left_move`**
- **`down_move`**
- **`rotate_left`**
- **`rotate_right`**

Partie 3 : Programmation du logiciel

3.1. Arborescence du projet

Le projet est développé en C++17 et s'appuie sur le framework Qt pour la gestion de l'interface graphique. L'organisation des fichiers suit une logique de spécialisation par classe :

Gestion de Projet :

- **`V3.pro`** : Fichier de configuration spécifiant les modules Qt (core, gui, widgets) et listant l'ensemble des fichiers sources, headers et formulaires du projet.

Cœur de Logique et Interface :

- **`main.cpp`**, **`mainwindow.cpp/h`** : Initialisation de l'application et gestion des états de l'interface (Menu, Jeu, Pause, Fin).
- **`game.cpp/h`**, **`grid.cpp/h`** : Moteur de jeu gérant la physique, les collisions et la grille de jeu.

Système de Pièces (tétrominos) :

- **`tetromino.cpp/h`** : Classe de base abstraite définissant les comportements communs (mouvement, rotation, accès aux coordonnées).
- Classes spécialisées : I_tetro, J_tetro, L_tetro, O_tetro, S_tetro, T_tetro, Z_tetro (headers et sources respectifs)

3.1.1. Constructeurs et destructeurs

On effectue l'allocation et la libération des variables avec `new` et `delete`. Ces opérateurs font appel aux constructeurs et destructeurs de chaque classe, qui initialisent et effacent les paramètres.

3.2. Structure de données et représentation

Le jeu repose sur une organisation matricielle pour l'espace de jeu et les pièces :

- **Grille** : La classe « Grid » gère un tableau dynamique d'entiers à deux dimensions : `int** g_grid`
  - 0 : Représente une case vide.
  - 1 à 7 : Représentent une case remplie. Le chiffre correspond à l'ID de la pièce d'origine (par exemple : 1 pour I, 2 pour J, etc.) et détermine la couleur lors du rendu.
- **Tétrominos** : Chaque pièce possède un tableau 3D `t_shape` de type : `std::array<std::array<std::array<int, 4>, 4>, 4>`
  - Ce tableau stocke les 4 phases de rotation.
  - Chaque phase est une matrice 4×4 où les valeurs non nulles dessinent la forme de la pièce.

3.2.1. Gestionnaires d'évènements

La navigation et les interactions sont gérées dans MainWindow via le système de *slots* de Qt :

- `on_play_button_clicked()` : Initialise l'objet Game et lance le chronomètre de jeu.
- `on_pause_button_clicked()` : Arrête le w_timer et bascule l'affichage sur l'écran de pause.
- `on_restart_button_clicked()` : Détruit l'instance actuelle de Game pour en créer une nouvelle, réinitialisant ainsi la grille et le score.
- `mousePressEvent(QMouseEvent* event)` : Détecte les clics sur la zone de jeu. Un clic gauche déclenche `rotate_tetro_left()` et un clic droit déclenche `rotate_tetro_right()`.

3.3. Points importants du code

3.3.1. Gestion des collisions

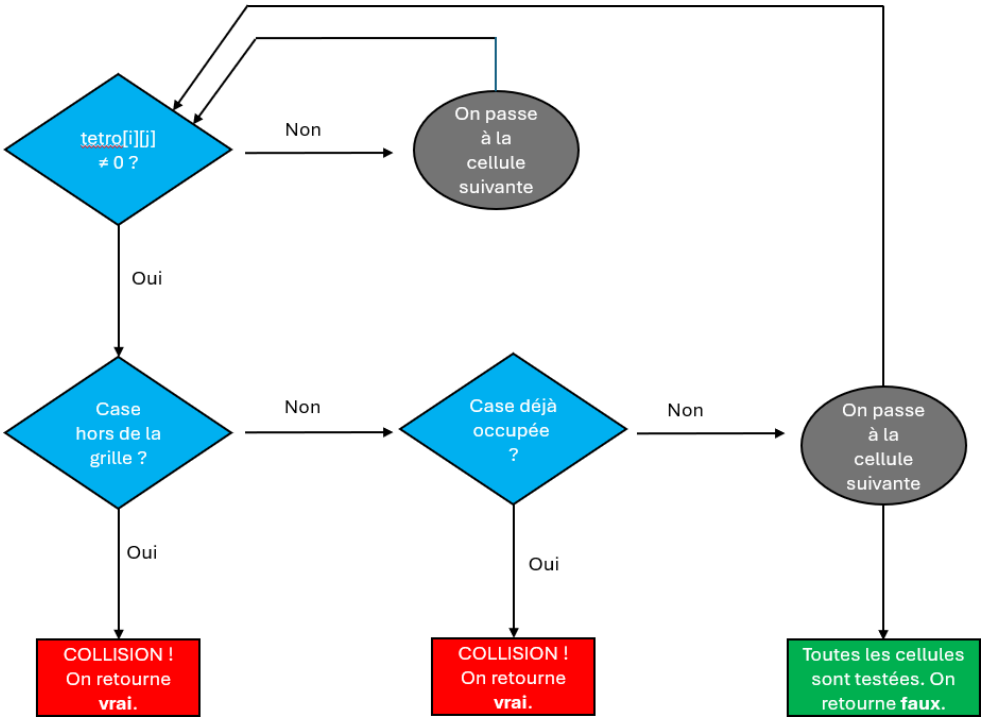
On utilise la fonction **`check_collision`** de la classe « Game ». Il s'agit d'une fonction booléenne qui renvoie `true` si elle détecte une collision, `false` sinon. On l'appelle avant que le tétromino n'effectue un mouvement ou une rotation.

Elle prend en paramètres un tableau 4×4 qui contient la forme du tétromino dans son état de rotation actuel, ainsi que la position `x_pos` et `y_pos` de son coin supérieur gauche. Dans la pratique, on passe en paramètre la position et la rotation hypothétiques après l'action. Si une collision est détectée, l'action n'a pas lieu. Si l'action était un mouvement vers le bas, cela signifie que le tétromino est arrivé au bas de la grille, et on applique alors les règles du jeu.

L'algorithme `check_collision` dans `game.cpp` est préventif. Avant chaque mouvement ou rotation, il parcourt la matrice 4×4 de la pièce et vérifie deux conditions pour toute cellule non vide :

1. **Limites** : Si `x_pos + j` est hors des bornes de la grille (0 à nbc) ou si `y_pos + i` dépasse nbl.
2. **Obstruction** : Si la case correspondante dans la grille est aussi différente de 0.

Si on a parcouru toutes les cases sans problème, on renvoie `false`.



Algorithme de gestion des collisions

3.3.2. Gestion de l'intégration

On utilise la fonction **`add_grid`** de la classe « Grid ». Cette fonction parcourt le tétromino sur le point d'être détruit et copie ses valeurs aux emplacements correspondant dans la grille.

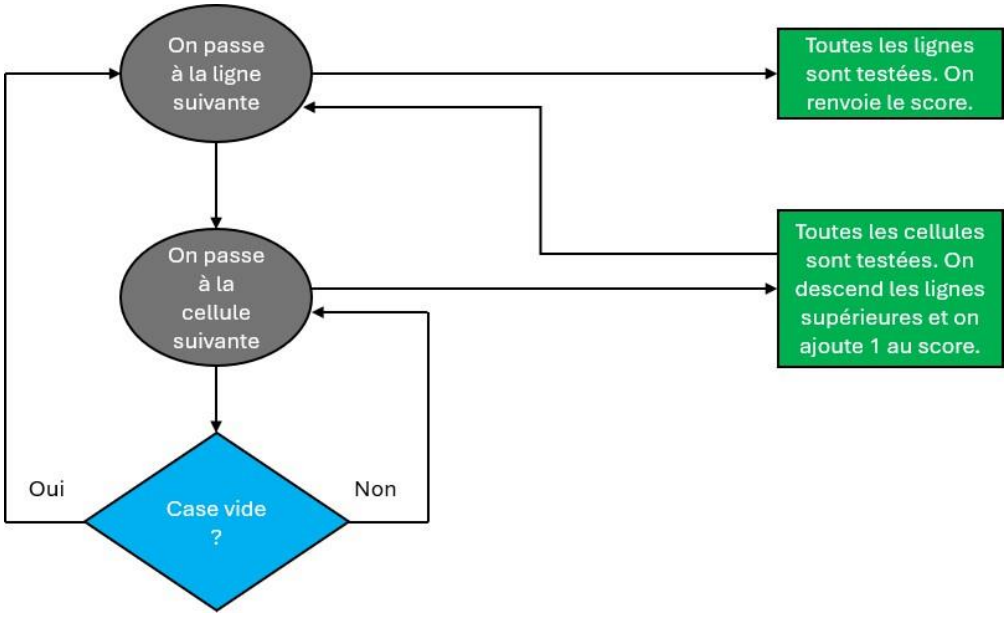
3.3.3. Gestion des suppressions de lignes

On utilise la fonction **`check_full_line`** de la classe « Grid ». Elle a quatre utilités majeures :

1. Détecter les lignes complètes
2. Supprimer les lignes complètes
3. Faire descendre les lignes situées au-dessus
4. Renvoyer le nombre de lignes supprimées pour mettre à jour le score

Elle renvoie donc un nombre entier entre 0 et 4.

Dans le détail, une première boucle de la fonction parcourt toutes les lignes de la grille en commençant par le bas. Pour chaque ligne, une autre boucle parcourt toutes les cases à la recherche d'une case vide. Si elle n'en trouve pas, elle en déduit que la ligne est pleine. Dans ce cas, une boucle parcourt toutes les lignes situées au-dessus de la ligne détectée pleine et en copie le contenu dans la ligne immédiatement inférieure. Cela permet d'effectuer les tâches 2 et 3 en une seule fois. On répète l'opération pour chaque ligne pleine détectée. On met à jour la variable `full_row_nbr` qui compte le nombre de lignes pleines (le maximum en une seule fois étant de 4).



Algorithme de gestion des suppressions de lignes

3.3.4. Gestion de l'ombre

On utilise la fonction **`display_projection`** de la classe « Game » qui prend en paramètre une copie du tétromino courant. Elle la fait descendre jusqu'à la ligne sans collision la plus basse possible, puis elle l'affiche en gris (pour ne pas confondre l'ombre avec les vrais tétrominos).

3.3.5. Gestion des sauvegardes

On utilise la fonction **`save`** de la classe « Game ». Elle enregistre la partie en cours sous la forme d'un fichier texte. Elle prend en paramètre le chemin (`path`) vers un dossier de sauvegarde.

Il faut donc vérifier avant le lancement du jeu la validité du chemin enregistré dans le code, et si nécessaire, le modifier. La même précaution s'applique aux **fichiers musicaux**. En cas d'arborescence erronée, un message d'erreur apparaîtra dans la console, mais l'application ne sera pas interrompue.

Lorsqu'on clique sur le bouton `continue`, la fonction **`on_continue_button_clicked`** se charge de récupérer et de lire le fichier texte pour restaurer la partie dans son état d'origine. Si on ne parvient pas à accéder au fichier de sauvegarde (que ce soit à cause d'une erreur ou parce qu'aucune partie n'a encore été enregistrée), un message d'erreur apparaîtra dans la console, et une nouvelle partie sera lancée. On ne peut sauvegarder qu'une partie à la fois.